

Mobile Hybrid Application

built using:

- Ionic
- Angular
- TypeScript
- Capacitor

What This Project REALLY Is

This is basically:

Railway GPS Navigation + Voice Alert Android App

NOT an embedded firmware project.

NOT a Spring Boot backend project.

NOT a Java Swing desktop app.

Actual Architecture

```
Android Tablet
  ↓
Ionic App
  ↓
Angular Components
  ↓
GPS Tracking
  ↓
Route Engine
  ↓
Voice Announcements
  ↓
Offline Storage
```

Huge Important Realization

You were earlier thinking:

Java + SQLite + Embedded System

But actual project stack is:

`Ionic + Angular + Capacitor + Android`

This means:

- frontend-heavy application
- mobile app architecture
- TypeScript-based logic
- native mobile integrations

Actual Tech Stack Used

From your PDF:

Area	Technology
UI	Ionic
Frontend	Angular
Language	TypeScript
Mobile Runtime	Capacitor
GPS	Capacitor Geolocation
Voice	Capacitor TTS
Audio	Native Audio Plugin
Storage	Ionic Storage
Platform	Android

So What is Ionic?

Ionic lets developers build Android/iOS apps using:

- HTML
- CSS
- TypeScript

instead of native Java/Kotlin.

Think of it like:

`Angular Website`
+
`Mobile APIs`
=
`Android App`

What Capacitor Does

Capacitor bridges web code with native Android features.

Example:

GPS

Audio

Storage

Permissions

TTS

all accessible from TypeScript.

Main Core Component

The most important file is:

`dashboard-v2.component.ts`

This is basically the heart of FogPASS.

It handles:

- GPS tracking
- speed calculation
- station detection
- audio announcements
- distance calculations

Core Flow of THIS Actual App

Step 1 — App Starts

Android tablet launches Ionic app.

Step 2 — User Selects Route

Route loaded from:

- CSV
- Excel

Step 3 — GPS Tracking Starts

Using:

@capacitor/geolocation

App continuously reads:

- latitude
- longitude
- speed

Step 4 — GPS Noise Filtering

This is VERY important.

Real GPS is noisy.

So they implemented filtering:

```
maxJumpMeters = 500  
maxJumpMetersPerSec = 200
```

This prevents fake GPS jumps.

Step 5 — Speed Calculation

Uses:

Exponential Moving Average (EMA)

```
Smooth speed values  
Reduce fluctuations
```

Step 6 — Distance Calculation

Uses:

Haversine Formula

to calculate earth distance between:

Current GPS
↓
Upcoming Signal

Step 7 — Station Detection

Core logic:

If train enters threshold distance
↓
Trigger announcement

Step 8 — Voice Announcement

Uses:

@capacitor-community/text-to-speech

Example:

"Upcoming station Vijayawada"

Step 9 — Offline Storage

Uses:

@ionic/storage-angular

to store:

- routes
- settings
- logs
- cached data

This is where SQLite-like storage comes indirectly.

NEW Correct Roadmap

Phase 1 — Web Basics

Learn:

- HTML
 - CSS
 - JavaScript
-

Phase 2 — TypeScript

MOST IMPORTANT.

Because entire project logic is in TypeScript.

Learn:

- interfaces
- classes
- async/await
- observables
- modules

Phase 3 — Angular

Critical.

Learn:

- components
 - services
 - routing
 - dependency injection
 - RxJS
 - lifecycle hooks
-

Phase 4 — Ionic

Learn:

- pages
- navigation
- mobile UI
- Ionic components

Phase 5 — Capacitor

Learn:

- GPS plugin
- permissions
- native APIs
- Android integration

Phase 6 — GPS Concepts

Learn:

- Haversine formula
- coordinate systems
- GPS noise
- filtering
- thresholds

Phase 7 — Real-Time Logic

Learn:

- intervals
- timers
- background tasks
- RxJS streams

Actual Project Structure

```
src/  
├── app/  
│   ├── login/  
│   ├── selectionv2/  
│   ├── route-management/  
│   ├── developer-options/  
│   ├── home/  
│   │   ├── dashboard-v2/  
│   │   ├── report-issue/  
│   │   └── update-route/
```

This is standard Angular/Ionic architecture.

Most Important Files

1. dashboard-v2.component.ts

Heart of application.

Contains:

- GPS logic
- alerts
- distance tracking
- announcements

2. voice-notification.service.ts

Handles TTS.

3. storage.service.ts

Handles offline data.

4. queue.service.ts

Handles background processing.

Biggest Technical Challenge

NOT UI.

NOT storage.

The hardest part is:

GPS filtering and station detection.

That is the actual engineering logic.

What You Should Build First

Step 1

Build simple Ionic app.

Step 2

Show current GPS location.

Step 3

Calculate speed.

Step 4

Load CSV route file.

Step 5

Calculate nearest station.

Step 6

Trigger voice alert.

Your BEST Learning Sequence

1. JavaScript
2. TypeScript
3. Angular
4. Ionic
5. Capacitor
6. GPS logic
7. RxJS

8. Mobile app debugging
9. Android build process

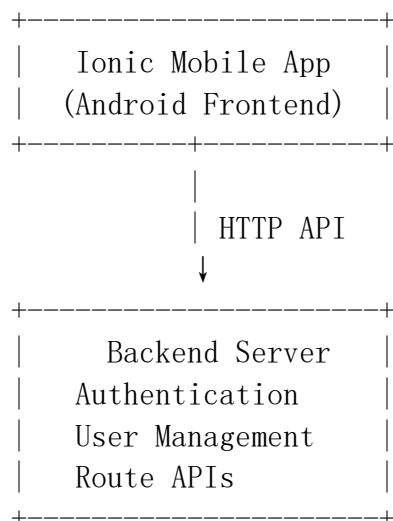
1. Mobile App (Frontend)

This Ionic + Angular project.

AND

2. Separate Server Code (Backend)

because login and some APIs depend on backend.



What You Should Do NOW

Step 1 — Run Backend

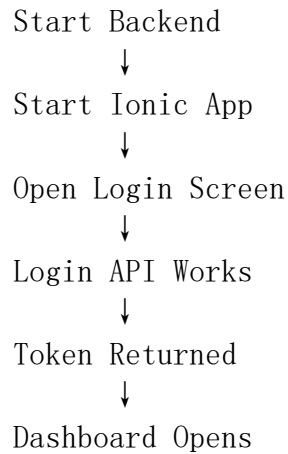
First identify backend technology.

Open backend folder.

See if it contains:

File	Meaning
------	---------

File	Meaning
pom.xml	Spring Boot Java
package.json	Node.js backend
requirements.txt	Python
build.gradle	Gradle Java
src/main/java	Spring Boot



Your project is actually:

Hybrid Architecture

Backend Responsibilities

Backend	Purpose
Login	Authentication
Token generation	Security
User profiles	User management
Route sync	Data management
Reports	Analytics

Mobile App Responsibilities

Mobile App	Purpose
GPS tracking	Real-time train tracking

Mobile App	Purpose
Distance calculations	Signal detection
Voice alerts	Announcements
Offline logic	Runtime operation
UI rendering	Dashboard

So Which Part Is REAL FogPASS Core?

Actually:

Frontend mobile app.

Because railway logic lives there.

COMPLETE PROJECT STRUCTURE

Your project actually has:

PART 1 — MOBILE APP

`Ionic + Angular`

This is:

- Android application
- UI
- GPS engine
- voice alerts
- offline logic

Folder

`src/app/`

Main Responsibility

Runs INSIDE Android tablet.

Feature	Purpose
Login UI	User login
Route selection	Choose train route
GPS tracking	Real-time location
Distance calculations	Signal detection
Voice announcements	Audio alerts
Offline storage	Local persistence
Dashboard	Real-time monitoring

PART 2 — BACKEND SERVER

Node.js + Express

This is:

- API server
- authentication system
- user management
- admin data

Folder

SERVER CODE/

Main Responsibility

Handles:

- login
- user profiles
- route management
- trip history
- admin operations

Frontend sends HTTP request



Express route receives request



Database queried



Response returned

PART 3 — DATABASE LAYER

Sequelize Models

Folder

db configuration/

Main Responsibility

Defines database tables.

Equivalent to:

@Entity in Spring Boot

Example Tables

File	Table
Users.js	Users
Trips.js	Trips
Routes.js	Routes
Sections.js	Railway sections
TripSignalDetails.js	Signal crossing logs

PART 4 — DATABASE

Your project uses:

Microsoft SQL Server

Database Stores

Data	Purpose
Users	Authentication
Trips	Journey logs
Routes	Railway route data
Signals	Signal coordinates
Issues	Problem reports

Utility Files

Contains:

- Haversine formula
- GPS calculations
- filtering

Understand:

- Haversine formula
- EMA filtering
- distance calculations